

Nama : Rika bagus Wijaya
Nim : G.231.24.0001
Matkul : Analisis kompleksitas algoritma
Praktikum 4

TENTUKAN KOMPLEKSITAS WAKTU BIG O DAN ANALISA CASE ALGORITMA PENCARIAN

1. Linear Search

Linear Search

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

contoh

```
data = [10, 20, 30, 40, 50]  
print(linear_search(data, 30)) # output: 2
```

Analisa Kompleksitas

- Best Case : $O(1)$
- Average Case : $O(n)$
- Worst Case : $O(n)$

Penjelasan

- Best Case terjadi ketika data yang dicari berada di indeks pertama.
- Average Case terjadi ketika data ditemukan di tengah array.
- Worst Case terjadi ketika data berada di akhir array atau tidak ditemukan sama sekali sehingga seluruh elemen harus dicek satu per satu.

2. Binary Search (Iteratif)

Binary Search (Iteratif)

```
def binary_search(arr, target):  
    left, right = 0, len(arr) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:
```

```

        left = mid + 1
    else:
        right = mid - 1

    return -1

# contoh (data harus terurut)
data = [10, 20, 30, 40, 50]
print(binary_search(data, 40)) # output: 3

```

Analisa Kompleksitas

- Best Case : $O(1)$
- Average Case : $O(\log n)$
- Worst Case : $O(\log n)$

Penjelasan

- Best Case terjadi ketika data yang dicari langsung ditemukan di posisi tengah.
- Average Case terjadi karena jumlah pencarian selalu dibagi dua setiap iterasi.
- Worst Case terjadi ketika pencarian dilakukan terus hingga ruang pencarian tersisa satu elemen.
- Algoritma ini hanya dapat digunakan pada data yang sudah terurut.

3. Binary Search (Rekursif)

Binary Search (Rekursif)

```

def binary_search_recursive(arr, left, right, target):
    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, right, target)
    else:
        return binary_search_recursive(arr, left, mid - 1, target)

# contoh
data = [10, 20, 30, 40, 50]
print(binary_search_recursive(data, 0, len(data)-1, 20)) # output: 1

```

Analisa Kompleksitas

- Best Case : $O(1)$
- Average Case : $O(\log n)$

- Worst Case : $O(\log n)$

Penjelasan

- Cara kerja sama seperti Binary Search Iteratif, namun menggunakan fungsi rekursif.
- Setiap pemanggilan fungsi membagi array menjadi dua bagian.
- Best Case terjadi saat data langsung ditemukan di tengah.
- Worst Case terjadi ketika proses pembagian dilakukan terus sampai data ditemukan atau data tidak ada.
- Data harus dalam keadaan terurut.

4. Jump Search

Jump Search

import math

def jump_search(arr, target):

 n = len(arr)

 step = int(math.sqrt(n))

 prev = 0

lompat blok

while arr[min(step, n) - 1] < target:

 prev = step

 step += int(math.sqrt(n))

if prev >= n:

return -1

linear search di blok

for i **in** range(prev, min(step, n)):

if arr[i] == target:

return i

return -1

contoh

data = [10, 20, 30, 40, 50, 60, 70]

print(jump_search(data, 60)) *# output: 5*

Analisa Kompleksitas

- Best Case : $O(1)$
- Average Case : $O(\sqrt{n})$
- Worst Case : $O(\sqrt{n})$

Penjelasan

- Algoritma melakukan lompatan sejumlah $\sqrt{n}$ untuk mempercepat pencarian.

- Setelah blok ditemukan, dilakukan Linear Search pada blok tersebut.
 - Best Case terjadi saat data langsung ditemukan pada lompatan pertama.
 - Worst Case terjadi saat data berada di akhir blok atau tidak ditemukan.
 - Data harus terurut agar Jump Search dapat bekerja dengan baik.
-

5. Interpolation Search

Interpolation Search

```
def interpolation_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high and target >= arr[low] and target <= arr[high]:
        if low == high:
            if arr[low] == target:
                return low
            return -1

        # rumus interpolasi
        pos = low + ((target - arr[low]) * (high - low) //
                    (arr[high] - arr[low]))

        if arr[pos] == target:
            return pos
        elif arr[pos] < target:
            low = pos + 1
        else:
            high = pos - 1

    return -1

# contoh
data = [10, 20, 30, 40, 50]
print(interpolation_search(data, 50)) # output: 4
```

Analisa Kompleksitas

- Best Case : $O(1)$
- Average Case : $O(\log \log n)$
- Worst Case : $O(n)$

Penjelasan

- Interpolation Search memperkirakan posisi data menggunakan rumus interpolasi.
- Algoritma bekerja sangat cepat jika data terdistribusi merata.
- Best Case terjadi saat posisi data langsung ditemukan.
- Average Case lebih cepat dibanding Binary Search pada data yang tersebar merata.

- Worst Case terjadi jika distribusi data tidak merata sehingga pencarian menjadi lambat.
- Data harus terurut.

Kesimpulan

Algoritma	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search Iteratif	$O(1)$	$O(\log n)$	$O(\log n)$
Binary Search Rekursif	$O(1)$	$O(\log n)$	$O(\log n)$
Jump Search	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
Interpolation Search	$O(1)$	$O(\log \log n)$	$O(n)$

Kesimpulan Akhir

- Linear Search cocok digunakan pada data kecil atau data yang belum terurut.
- Binary Search lebih cepat dibanding Linear Search, tetapi membutuhkan data terurut.
- Jump Search lebih efisien dibanding Linear Search pada data terurut.
- Interpolation Search sangat cepat pada data yang distribusinya merata.
- Binary Search dan Interpolation Search merupakan algoritma pencarian yang lebih optimal untuk data besar.